Regression

in Machine Learning

10 types • diagrams • examples • Colab code • projects

Concepts | Diagrams | Code | Projects

Data Science Journey • Part 5 →

What is Regression?

Predicting a CONTINUOUS number

A supervised learning technique that models the relationship between input features (X) and a numeric target (y) to predict new values.

Regression → number

house price, salary, temperature, sales

Classification → category

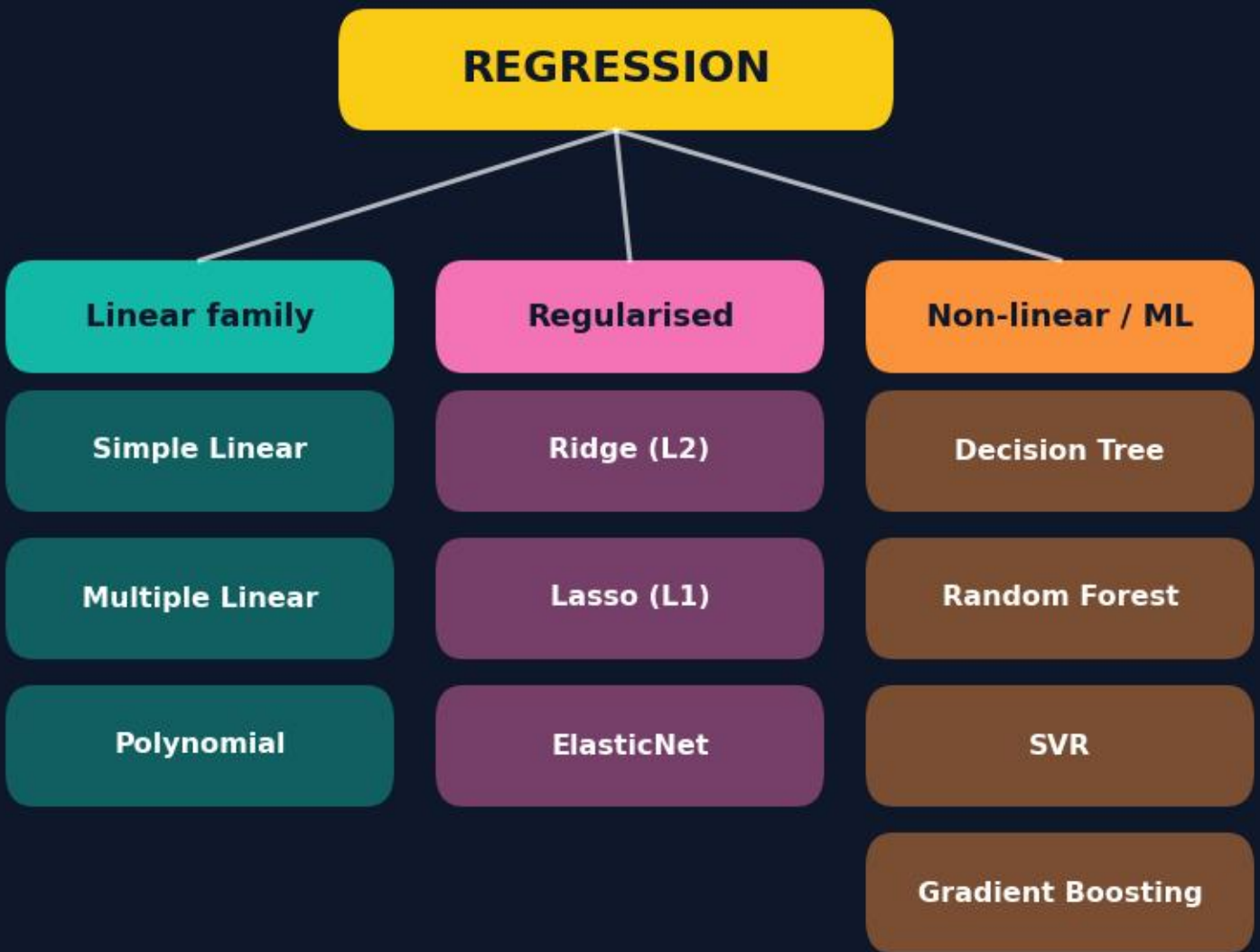
spam / not spam, disease yes / no

Key terms

target (y) • features (X) • coefficients • residual = actual − predicted

Goal: find the line / curve that minimises the prediction error

Types of Regression: Map



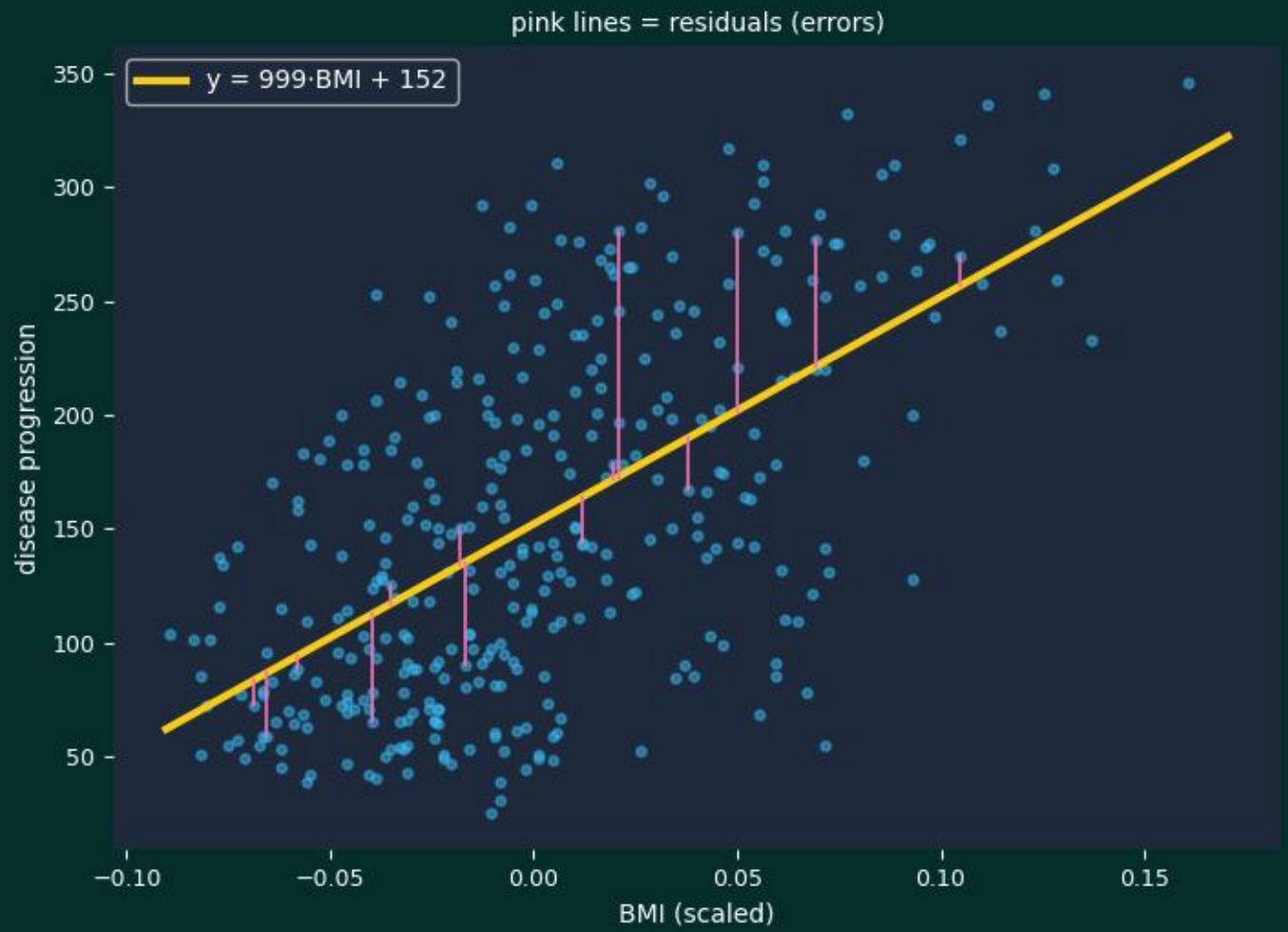
⚠ Logistic regression has 'regression' in its name
but predicts categories, so it is a CLASSIFICATION model.

1. Simple Linear

One feature predicts the target using a straight line.
Slope = effect of x, intercept = value when x = 0.

$$y = m \cdot x + c$$

Example: BMI → diabetes progression ($R^2 = 0.23$ on test data)

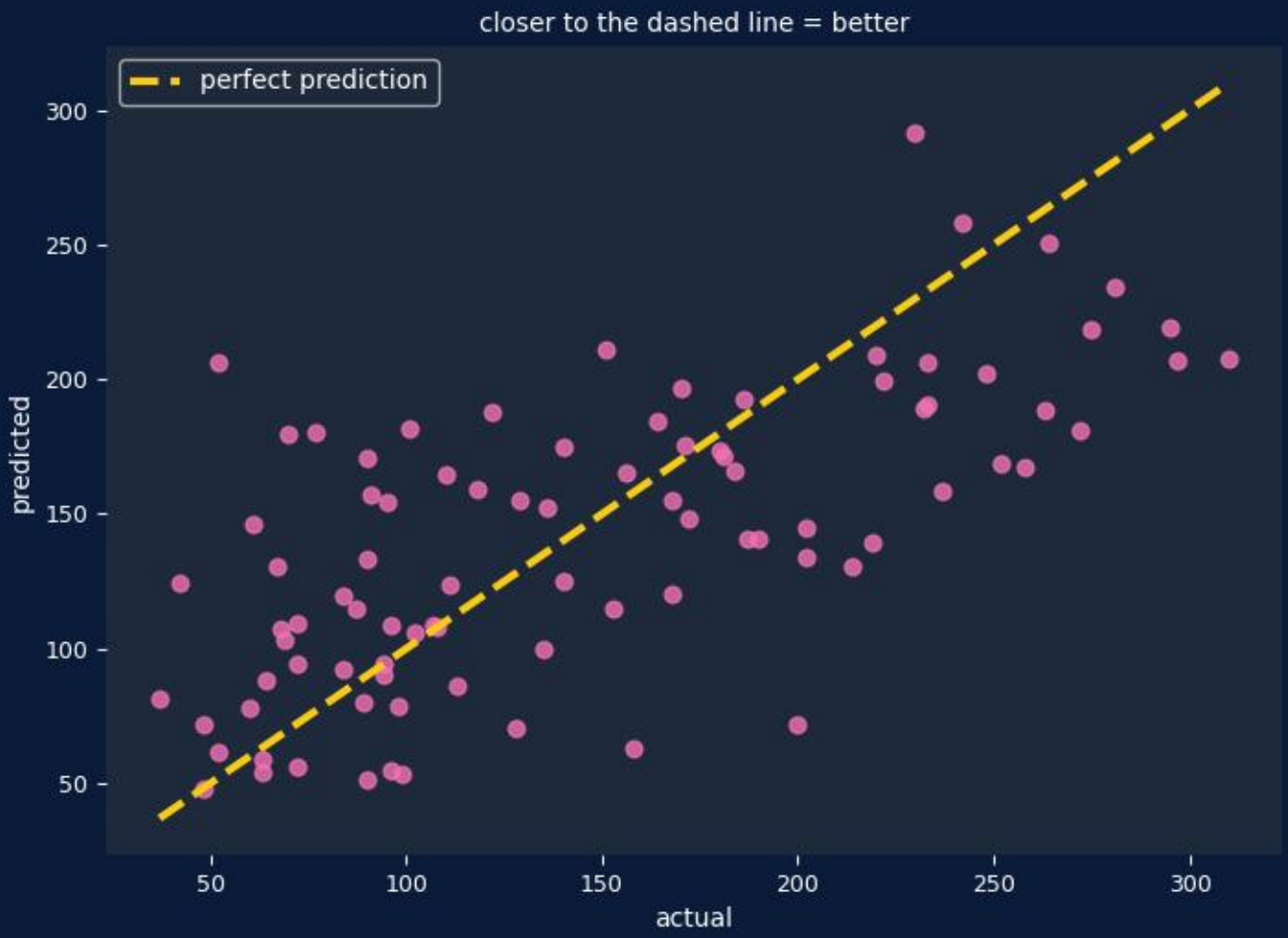


2. Multiple Linear

Many features predict the target. Each feature gets its own coefficient (weight) in one flat plane.

$$y = b0 + b1 \cdot x1 + b2 \cdot x2 + \ldots + bn \cdot xn$$

Example: 10 body measures → progression ($R^2 = 0.45$, RMSE = 53.9)

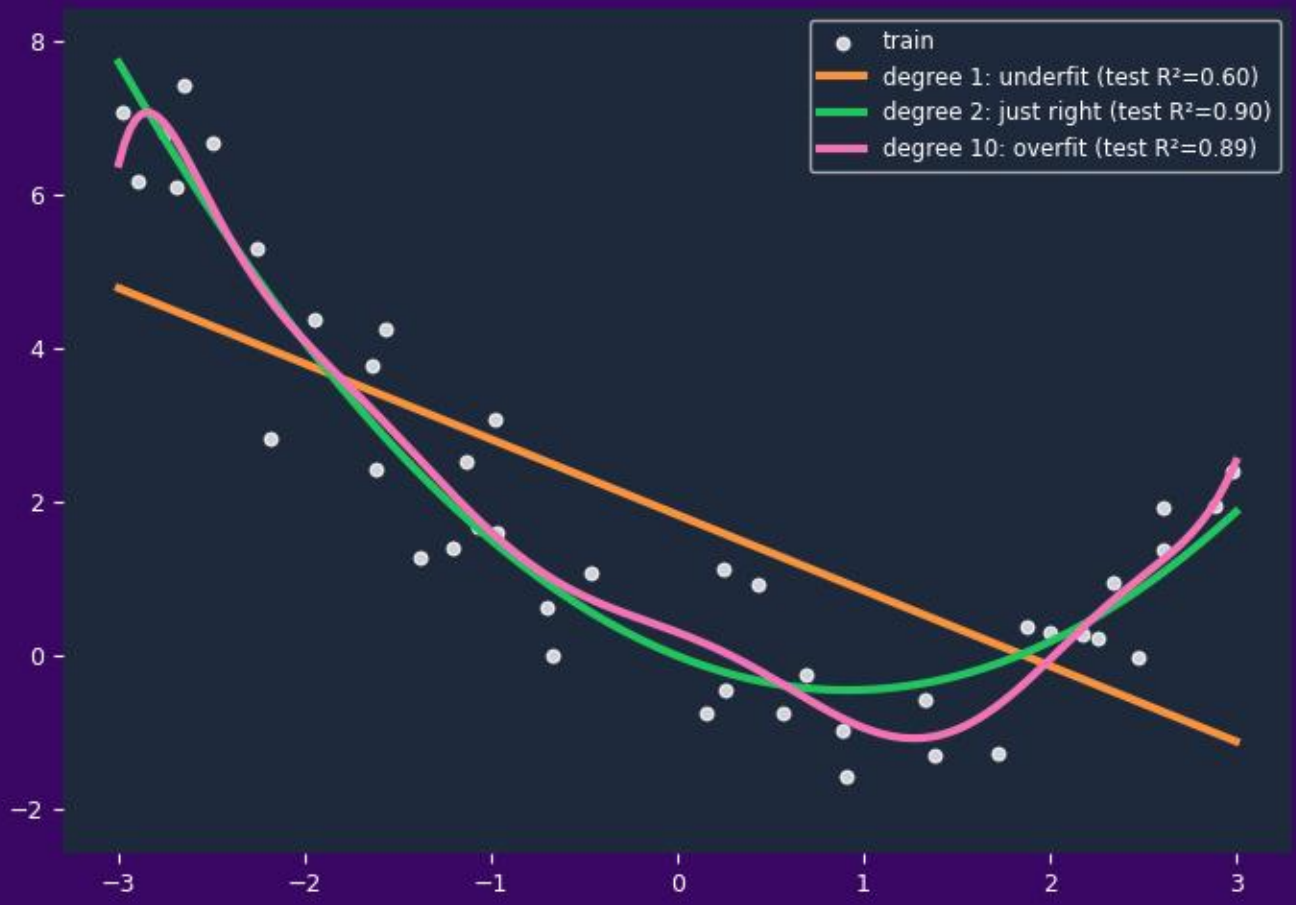


3. Polynomial

Still a linear model, but features are raised to powers (x^2, x^3 ...) so it can fit CURVES.
Too high a degree = overfitting.

$y = b0 + b1 \cdot x + b2 \cdot x^2 + \ldots + bd \cdot x^d$

Example: growth curves, speed vs fuel use, dose vs response



4-6. Regularised Regression

Add a penalty to stop coefficients getting too big

Ridge (L2)

penalty = α·Σ b^2 shrinks all coefficients,
never exactly zero. Good for correlated features.

Lasso (L1)

penalty = α·Σ |b| can shrink coefficients to ZERO,
so it also does feature selection.

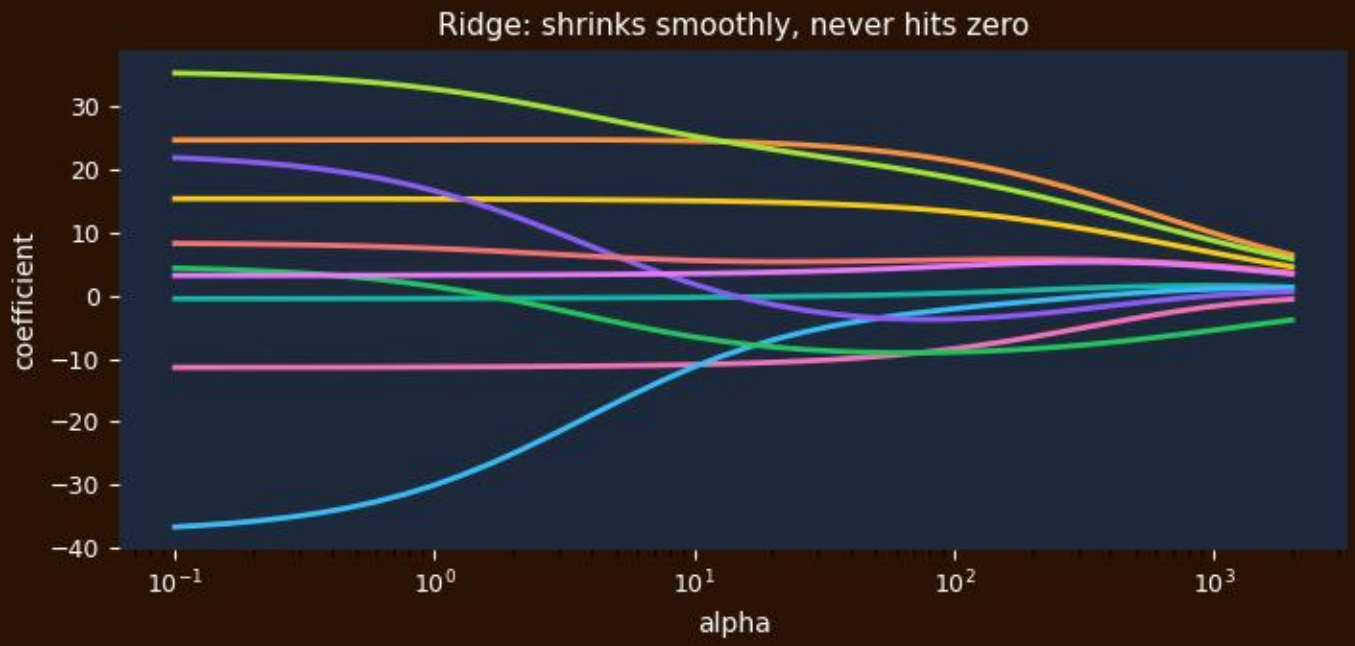
ElasticNet (L1 + L2)

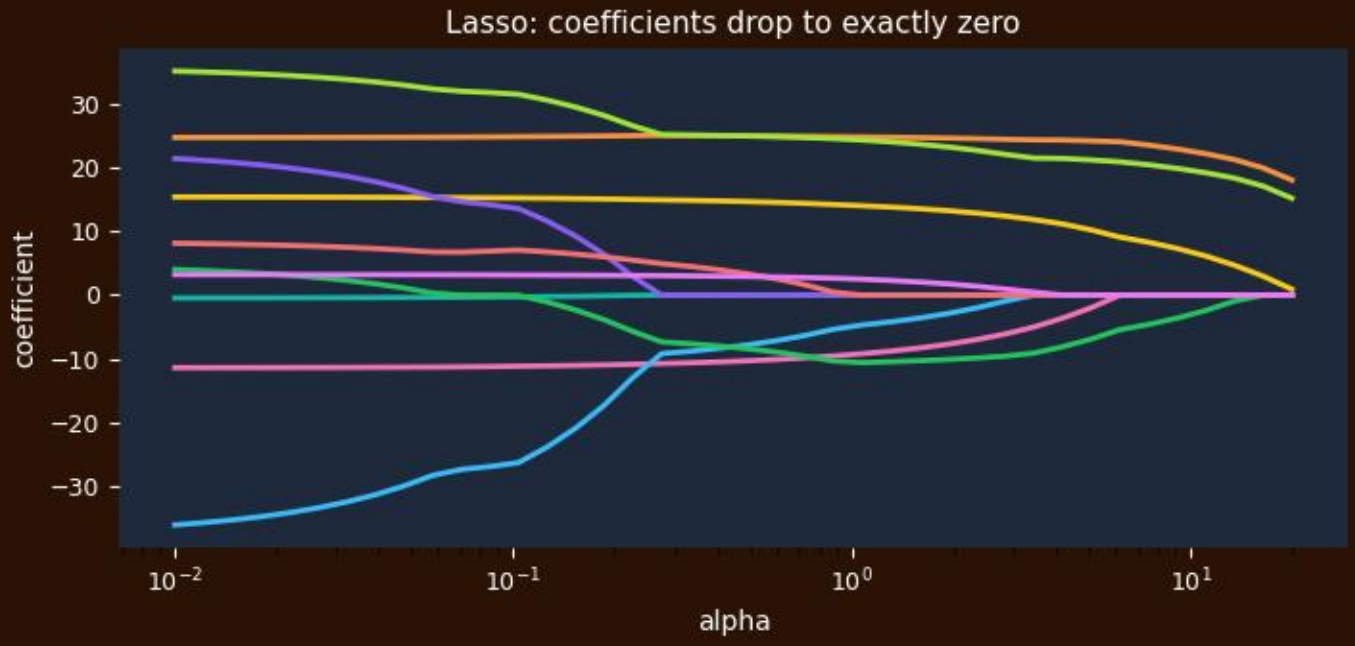
mixes both penalties (l1_ratio). Best when many
features are correlated AND some are useless.

On Diabetes data (alpha = 0.5): Lasso zeroed 1 features,
ElasticNet zeroed 0. Scale features first!

Coefficient Shrinkage

Bigger alpha = stronger penalty





7-10. Non-linear Regressors

When a straight line is not enough

Decision Tree

splits data into boxes; predicts the average of each box

Random Forest

many trees on random samples; averages them (stable)

SVR

fits a tube (epsilon margin) around data; kernels bend it

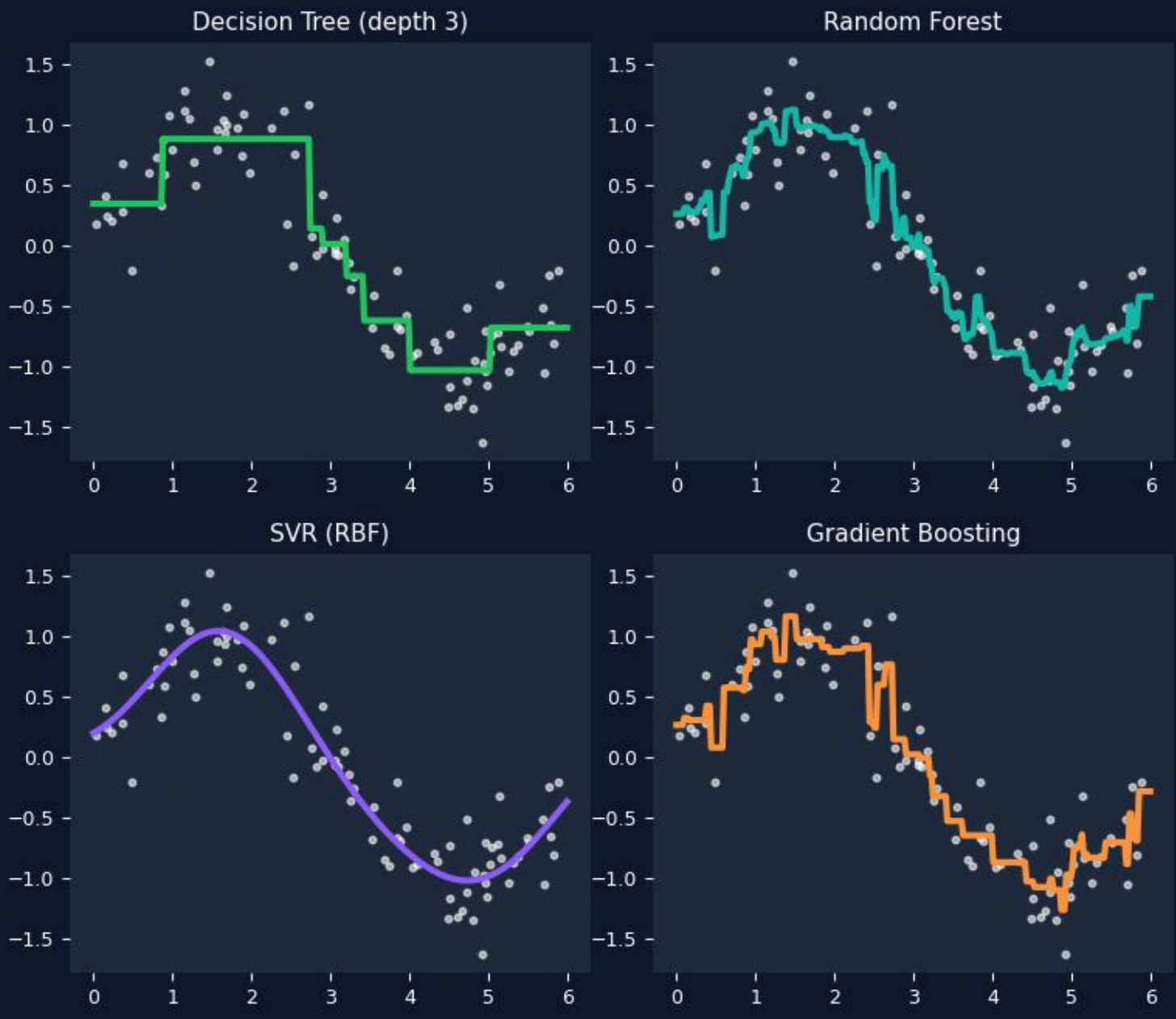
Gradient Boosting

trees built one after another, each fixing the last one's errors

Tip: tree models need no scaling; SVR does.

Non-linear Fits: Diagram

Same wavy data, four different models



Model Showdown

Test R^2 on the Diabetes dataset (442 patients)



Best here: SVR (RBF) (R^2 0.51). Teal = linear, pink = regularised, orange = ML models.

Code 1: Linear

Google Colab • Simple + Multiple

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

df = load_diabetes(as_frame=True).frame   # 442 rows
X, y = df.drop(columns="target"), df["target"]

# ---- 1. SIMPLE linear regression (1 feature) ----
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=42)
m = LinearRegression().fit(Xtr[["bmi"]], ytr)
print("slope:", m.coef_[0], "intercept:", m.intercept_)
print("R2:", r2_score(yte, m.predict(Xte[["bmi"]])))
plt.scatter(Xte["bmi"], yte)
plt.plot(Xte["bmi"], m.predict(Xte[["bmi"]]), "r")

# ---- 2. MULTIPLE linear regression (all features) ----
mlr = LinearRegression().fit(Xtr, ytr)
pred = mlr.predict(Xte)
print("R2  :", r2_score(yte, pred))
print("RMSE:", mean_squared_error(yte, pred) ** 0.5)
print(dict(zip(X.columns, mlr.coef_.round(1))))
```

Code 2: Poly + Regularised

Google Colab • Polynomial, Ridge, Lasso, ElasticNet

```
from sklearn.preprocessing import (PolynomialFeatures,
                                   StandardScaler)
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import Ridge, Lasso, ElasticNet

# ---- 3. POLYNOMIAL regression ----
rng = np.random.default_rng(0)
x = np.sort(rng.uniform(-3, 3, 60)).reshape(-1, 1)
yp = 0.5 * x.ravel() ** 2 - x.ravel() + rng.normal(0, .8, 60)
xa, xb, ya, yb = train_test_split(x, yp, test_size=.3,
                                  random_state=1)
for d in [1, 2, 10]:
    poly = make_pipeline(PolynomialFeatures(d),
                         LinearRegression()).fit(xa, ya)
    print("degree", d, "test R2:", round(poly.score(xb, yb), 3))

# ---- 4-6. REGULARISED regression ----
models = {
 "Ridge (L2)": Ridge(alpha=1.0),
 "Lasso (L1)": Lasso(alpha=0.5),
 "ElasticNet": ElasticNet(alpha=0.5, l1_ratio=0.5)}
for name, reg in models.items():
    pipe = make_pipeline(StandardScaler(), reg)
    pipe.fit(Xtr, ytr)
    print(name, "R2:", round(pipe.score(Xte, yte), 3),
          "| zero coefs:", (pipe[-1].coef_ == 0).sum())
```

Code 3: Non-linear

Google Colab • Tree, Forest, SVR, Boosting

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (RandomForestRegressor,
                              GradientBoostingRegressor)
from sklearn.svm import SVR

# ---- 7-10. NON-LINEAR / ML regressors ----
models = {
 "Decision Tree": DecisionTreeRegressor(max_depth=4,
                                        random_state=42),
 "Random Forest": RandomForestRegressor(n_estimators=200,
                                        random_state=42),
 "SVR (RBF)": make_pipeline(StandardScaler(), SVR(C=100)),
 "Gradient Boosting": GradientBoostingRegressor(
                                        random_state=42)}
rows = []
for name, m in models.items():
    m.fit(Xtr, ytr); p = m.predict(Xte)
    rows.append([name, r2_score(yte, p),
                 mean_squared_error(yte, p) ** 0.5])
res = pd.DataFrame(rows, columns=["model", "R2", "RMSE"])
print(res.sort_values("R2", ascending=False).round(3))

# Tip: compare models fairly with cross-validation
from sklearn.model_selection import cross_val_score
print(cross_val_score(models["Random Forest"], X, y,
                      cv=5, scoring="r2").mean())
```

Metrics & Assumptions

How to judge a regression model

MAE

mean |actual − predicted| (easy to read, same unit as y)

RMSE

√ mean (error²) (punishes big errors more)

R^2

1 − SSres/SStot (1 = perfect, 0 = predicts the mean)

Linear regression assumptions (LINE)

L Linearity: y relates linearly to X
I Independence of errors
N Normal residuals E Equal variance (homoscedasticity)

Which One to Use?

Clear straight-line trend

Simple / Multiple Linear (start here)

Curved relationship

Polynomial (low degree) or tree-based models

Many correlated / useless features

Ridge, Lasso or ElasticNet

Complex, non-linear, tabular data

Random Forest or Gradient Boosting

Small data, smooth pattern

SVR with RBF kernel

Regression Projects (1/2)

Beginner to intermediate • search these on Kaggle / UCI

House Price Prediction

California Housing / Ames Housing • predict price

Salary Prediction

years of experience, role, education → salary

Used Car Price

brand, year, km, fuel → resale price (CarDekho)

Medical Insurance Cost

age, BMI, smoker → charges (Insurance dataset)

Student Score Prediction

study hours, attendance → marks (UCI Student)

Fuel Efficiency (MPG)

engine, weight, year → mpg (UCI Auto MPG)

Regression Projects (2/2)

Intermediate to advanced

Sales / Demand Forecasting

Walmart, Rossmann, Bike Sharing Demand

Airbnb Price Prediction

location, rooms, reviews → nightly price

Taxi Fare & ETA Prediction

distance, time, pickup zone → fare / duration

Crop Yield Prediction

rainfall, soil, temperature → yield per hectare

Air Quality (PM2.5) Prediction

weather + pollutants → PM2.5 level

Concrete Strength Prediction

mix ingredients → compressive strength (UCI)

Which regression project will you build first?

Comment your pick

Repost to help a learner

Follow: next is Classification

#DataScience #Regression #MachineLearning #Python #Projects